


SWIGGY

ADVANCED SQL PROJECT

Presented by : DIVYA SHARMA





ABOUT SWIGGY

Swiggy is one of India's leading food-delivery and quick-commerce platforms, offering fast and reliable delivery from restaurants, cloud kitchens, and grocery stores. Supported by a strong logistics network and user-friendly app, it provides convenience, real-time tracking, and a wide product range. The company also leverages data analytics to optimize delivery routes, forecast demand, personalize recommendations, and enhance overall operational efficiency, enabling faster and more informed decision-making.





SQL QUERIES ANALYSIS





SWIGGY



1. Display all customers who live in 'Delhi'

```
SELECT * FROM customers
WHERE city = 'delhi';
```

customer_id	name	email	phone_number	city	address
2	Rohini Verma	rohini.verma@yahoo.com	9823456789	Delhi	B-23, Saket
5	Manish Kumar	NULL	9834567890	Delhi	D-45, Lajpat Nagar
18	Sonali Mishra	NULL	9878345678	Delhi	N-54, Karol Bagh
NULL	NULL	NULL	NULL	NULL	NULL



The purpose of this SQL query is to retrieve all customer records whose city is 'Delhi' from the customers table.

2. Find the average rating of all restaurants in 'Mumbai'

```
SELECT
    city, AVG(rating)
FROM
    restaurants
WHERE
    city = 'MUMBAI';
```



city	AVG(rating)
Mumbai	4.300000

To calculate the overall average rating of all restaurants located in Mumbai, helping understand the general quality and customer satisfaction level in that city.



3. List all customers who have placed at least one order

```
SELECT DISTINCT
  name, customer_id
FROM
  customers
JOIN
  orders USING (customer_id)
```

name	customer_id
Amit Sharma	1
Rohini Verma	2
Rajesh Gupta	3
Sneha Mehta	4
Manish Kumar	5
Priya Singh	6
Vikas Reddy	7
Anjali Patel	8
Suresh Nair	9

This query retrieves all customers who have placed at least one order by joining the customers and orders tables on customer_id. DISTINCT ensures each customer appears only once.



4. Display the total number of orders placed by each customer.

```
SELECT
    name, customer_id, COUNT(order_id)
FROM
    customers
    JOIN
    orders USING (customer_id)
GROUP BY name , customer_id;
```

name	customer_id	count(order_id)
Amit Sharma	1	2
Rohini Verma	2	3
Rajesh Gupta	3	3
Sneha Mehta	4	2
Manish Kumar	5	4
Priya Singh	6	3
Vikas Reddy	7	3
Anjali Patel	8	3
Suresh Nair	9	1
Kavita Deshmukh	10	2
Vivek Bhatt	11	2
Meera Joshi	12	2
Pankaj Jain	13	2
Nidhi Saxena	14	3
Ashok Kumar	15	3

The purpose of this SQL query is to show each customer with the total number of orders they have placed. It does this by joining the customers and orders tables using customer_id and counting the orders for each customer.

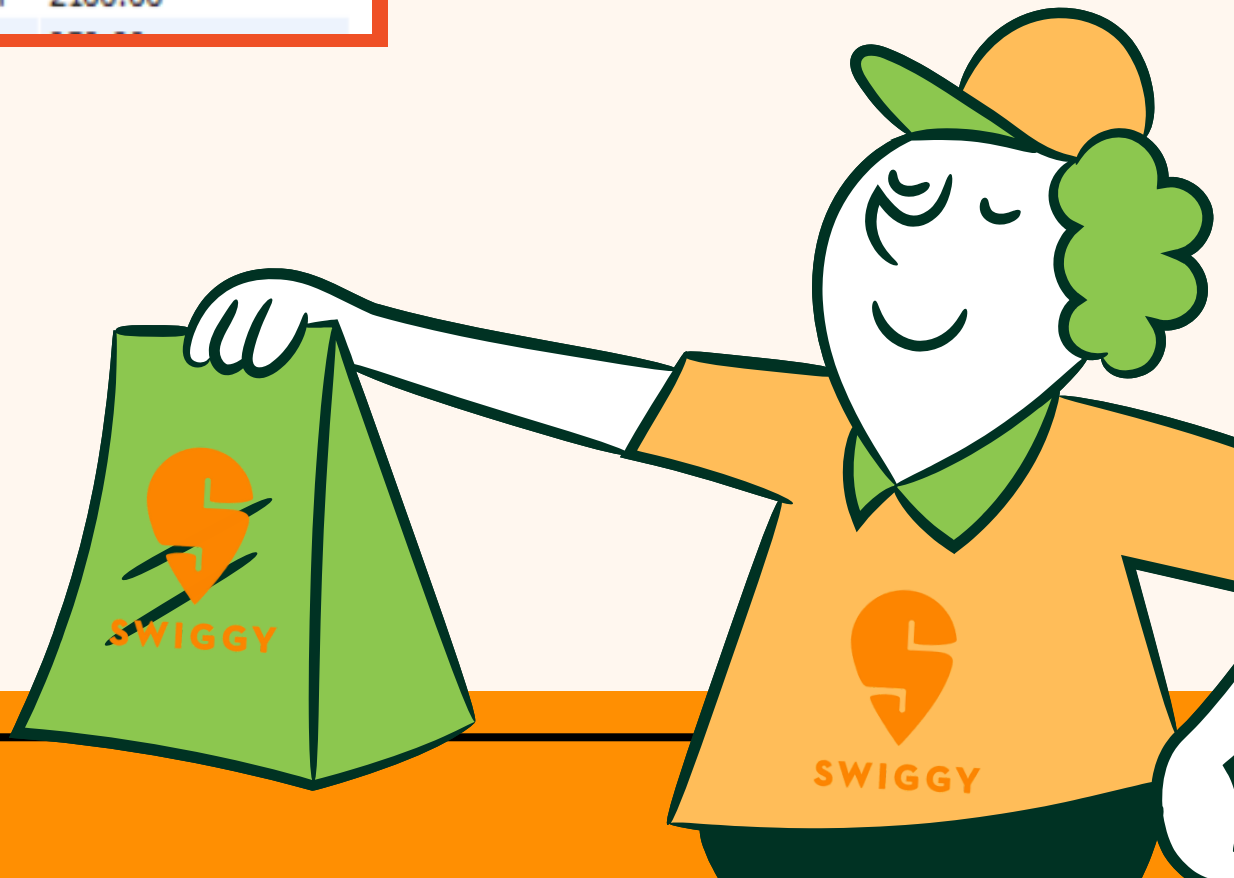


5. Find the total revenue generated by each restaurant.

```
SELECT
    restaurant_id, name, SUM(total_amount)
FROM
    restaurants
    JOIN
    orders USING (restaurant_id)
GROUP BY restaurant_id , name;
```

restaurant_id	name	sum(total_amount)
1	Spice of India	1100.00
2	Tandoori Flames	1200.00
3	Biryani House	5300.00
4	Curry Pot	3200.00
5	Taste of Punjab	600.00
6	Royal Biryani	650.00
7	Coastal Delight	2100.00
8	Veggie Delight	1600.00
9	Gujarat Express	2550.00
10	Andhra Spice	4050.00
11	Punjabi Tadka	900.00
12	Flavours of Ben...	4050.00
13	South Treat	2950.00
14	The Great India...	1600.00
15	Rajasthani Rasoi	2100.00

The purpose of this SQL query is to calculate how much total revenue each restaurant has generated.





SWIGGY

6. Find the top 5 restaurants with the highest average rating

```
SELECT
    *
FROM
    restaurants
ORDER BY rating DESC
LIMIT 5;
```

restaurant_id	name	city	cuisine	rating
3	Biryani House	Hyderabad	Hyderabadi	4.80
22	Paradise Biryani	Hyderabad	Biryani	4.80
30	Lucknowi Nawabi	Lucknow	Awadhi	4.70
6	Royal Biryani	Kolkata	Biryani	4.70
12	Flavours of Bengal	Kolkata	Bengali	4.60

It does this by sorting all restaurants in descending order of their rating and then using LIMIT 5 to return only the top five entries with the highest ratings.



7. Display all customers who have never placed an order

```
SELECT
    *
FROM
    customers
    LEFT JOIN
    orders USING (customer_id)
WHERE
    order_id IS NULL;
```

by performing a LEFT JOIN between the customers and orders tables.

- A LEFT JOIN returns all customers, along with matching order details if they exist.
- For customers who have no matching orders, the order_id will be NULL.

By filtering with WHERE order_id IS NULL, the query selects only those customers who have not placed any order.

customer_id	name	email	phone_number	city	address	order_id	restaurant_id	order_date	total_amount	status
24	Sonal Kaur	sonal.kaur@gmail.com	NULL	Amritsar	S-19, Ranjit Avenue	NULL	NULL	NULL	NULL	NULL
25	Vivek Malhotra	vivek.malhotra@hotmail.com	9812789012	Thane	NULL	NULL	NULL	NULL	NULL	NULL
26	Divya Iyer	NULL	9823890123	Bangalore	T-20, Indiranagar	NULL	NULL	NULL	NULL	NULL
27	Rakesh Yadav	rakesh.yadav@gmail.com	9834901234	Varanasi	U-31, Lanka	NULL	NULL	NULL	NULL	NULL
28	Mona Sharma	mona.sharma@yahoo.com	9845012345	Ranchi	NULL	NULL	NULL	NULL	NULL	NULL
29	Sudha Pillai	sudha.pillai@gmail.com	9856123789	Kozhikode	V-42, Mavoor Road	NULL	NULL	NULL	NULL	NULL
30	Gaurav Khanna	NULL	9867238901	Gwalior	W-53, City Centre	NULL	NULL	NULL	NULL	NULL





SWIGGY

8. Find the number of orders placed by each customer in 'Mumbai'

```
SELECT
    customer_id, name, city, COUNT(order_id)
FROM
    customers
    JOIN
    orders USING (customer_id)
WHERE
    city = 'mumbai'
GROUP BY customer_id , name , city;
```

customer_id	name	city	count(order_id)
1	Amit Sharma	Mumbai	2
3	Rajesh Gupta	Mumbai	3
19	Arjun Desai	Mumbai	2
23	Ravi Singh	Mumbai	2

The purpose of this query is to find how many orders each customer from Mumbai has placed. It filters customers based on the city 'Mumbai', joins them with their orders, and then uses COUNT(order_id) to calculate the total number of orders for each customer.



9. Display all orders placed in the last 30 days

```
SELECT
    *
FROM
    orders
WHERE
    order_date >= DATE_SUB((SELECT
        MAX(order_date)
    FROM
        orders),
        INTERVAL 30 DAY);
```

order_id	customer_id	restaurant_id	order_date	total_amount	status
1	1	3	2024-08-01 00:00:00	750.00	Completed
4	4	7	2024-08-01 00:00:00	850.00	Completed
11	10	12	2024-08-01 00:00:00	900.00	Completed
16	13	10	2024-08-01 00:00:00	950.00	Completed
22	17	1	2024-08-01 00:00:00	450.00	Completed
34	8	3	2024-08-01 00:00:00	1150.00	Completed
53	14	12	2024-08-01 00:00:00	1000.00	Processing
2	2	5	2024-08-02 00:00:00	600.00	Completed
9	8	6	2024-08-02 00:00:00	650.00	Completed
20	16	18	2024-08-02 00:00:00	850.00	Processing

It first finds the latest order_date using MAX(order_date) and then returns all orders whose dates fall within 30 days before that date. This helps in analyzing recent order activity within a rolling 30-day window.



10. List all delivery partners who have completed more than 1 delivery

```
SELECT
    partner_id, name, COUNT(order_delivery_id) AS delivery_count
FROM
    deliverypartners
    JOIN
    orderdelivery USING (partner_id)
GROUP BY partner_id , name
HAVING delivery_count > 1;
```

partner_id	name	delivery_count
1	Amit Sharma	2
2	Ravi Kumar	5
3	Priya Patel	3
4	Suresh Reddy	6
5	Anita Desai	4
6	Rajesh Gupta	4
7	Sonia Agarwal	3
8	Vikram Singh	2
9	Sneha Iyer	2
12	Reena Rao	2
13	Mohit Saini	2
16	Ritika Sharma	2

- FROM & JOIN: Links each delivery partner with their deliveries using partner_id.
- SELECT: Shows partner_id, name, and counts deliveries as delivery_count.
- GROUP BY: Counts deliveries per partner.
- HAVING: Keeps only partners with more than one delivery.



11. Find the customers who have placed orders on exactly three different days

```
SELECT
    customer_id,
    name,
    COUNT(DISTINCT DATE(order_date)) AS Order_days
FROM
    customers
    JOIN
    orders USING (customer_id)
GROUP BY customer_id , name
HAVING order_days = 3;
```

customer_id	name	Order_days
8	Anjali Patel	3
15	Ashok Kumar	3
14	Nidhi Saxena	3
6	Priya Singh	3
2	Rohini Verma	3
18	Sonali Mishra	3

- Joining the customers and orders tables to link each customer with their orders.
- Counting the number of distinct order dates for each customer.
- Filtering the results to include only those customers whose order count across different days is exactly three.





SWIGGY

12. Find the delivery partner who has worked with the most different customers

```
with t as (SELECT
    partner_id, name, count( distinct customer_id) as c_id
FROM
    deliverypartners
    JOIN
    orderdelivery USING (partner_id)
    JOIN
    orders USING (order_id)
GROUP BY partner_id , name)
select * from t
where c_id = (select max(c_id) from t);
```

partner_id	name	c_id
4	Suresh Reddy	6

The purpose of this SQL query is to find the delivery partner who has delivered orders to the highest number of different customers.

- Joining deliverypartners, orderdelivery, and orders to link each partner with the customers they served.
- Counting the distinct customers per partner.
- Sorting the results in descending order of customer count and selecting the top partner.



13. Identify customers who have the same city and have placed orders at the same restaurants, but on different dates

```
with t as
( select customer_id, restaurant_id, order_date, name, city
  from orders
  join customers
  using (customer_id))
select distinct *
from t t1
join t t2
where t1.city = t2.city and t1.restaurant_id = t2.restaurant_id
and t1.customer_id <> t2.customer_id and
t1.order_date <> t2.order_date and t2.customer_id < t1.customer_id;
```

customer_id	restaurant_id	order_date	name	city	customer_id	restaurant_id	order_date	name	city
18	3	2024-08-05 00:00:00	Sonali Mishra	Delhi	5	3	2024-08-04 00:00:00	Manish Kumar	Delhi
18	3	2024-08-05 00:00:00	Sonali Mishra	Delhi	5	3	2024-08-07 00:00:00	Manish Kumar	Delhi
23	8	2024-08-09 00:00:00	Ravi Singh	Mumbai	19	8	2024-08-03 00:00:00	Arjun Desai	Mumbai

The purpose of this query is to identify pairs of customers who live in the same city, ordered from the same restaurant, but on different dates.





SWIGGY

THANK YOU

